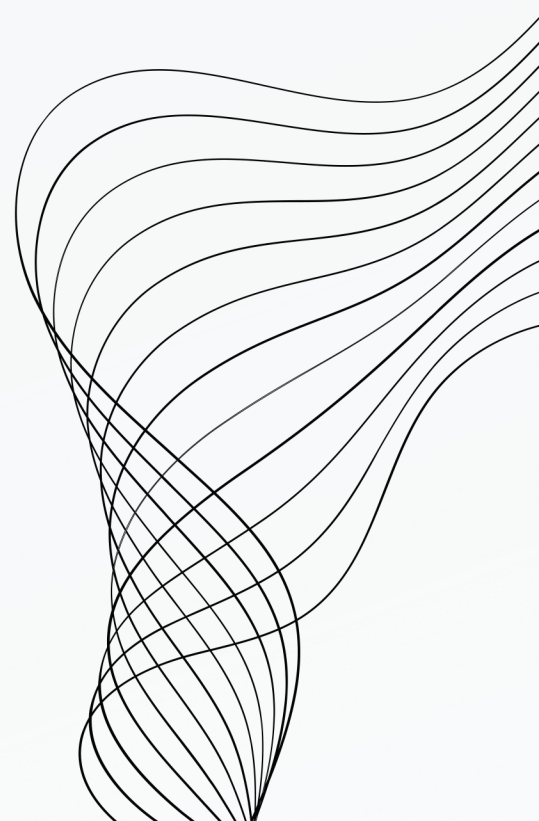
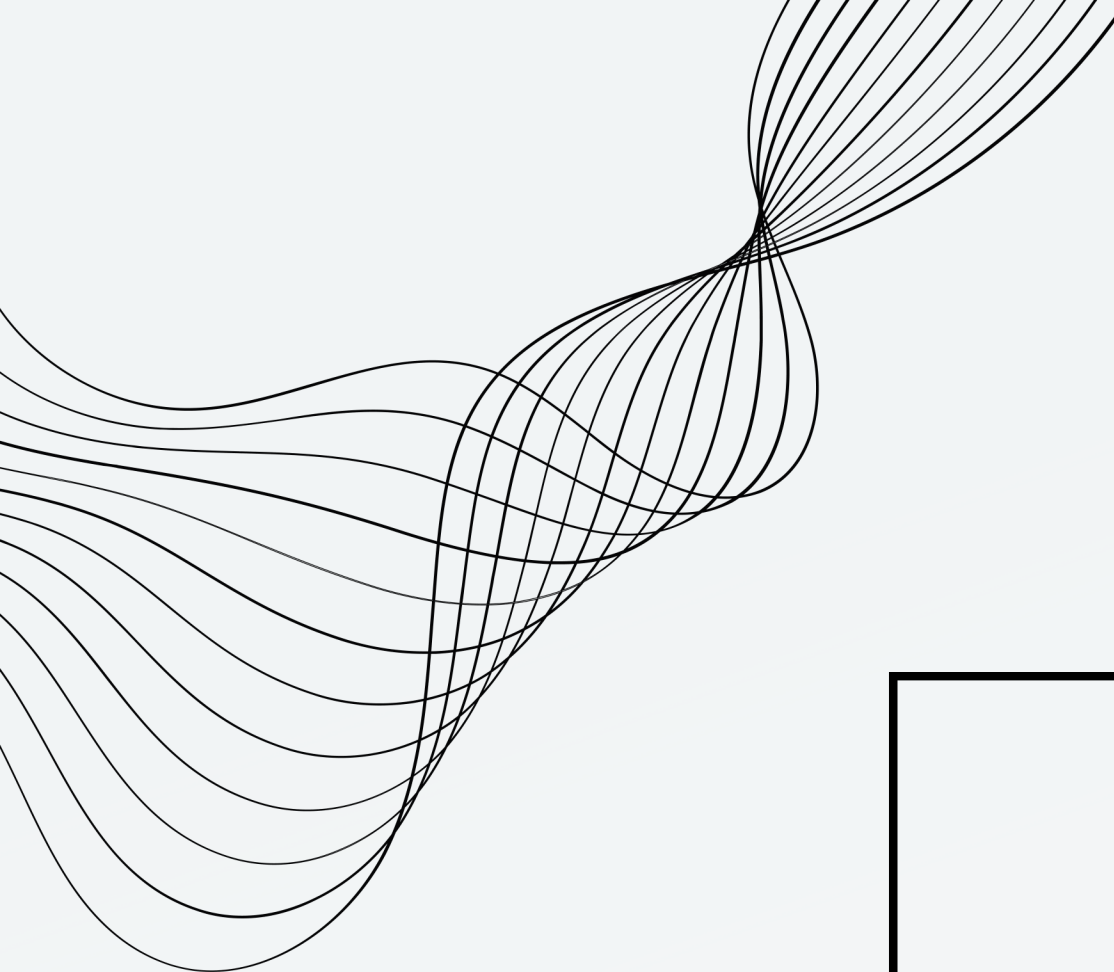




DATA SCIENCE

EXPOSYS DATA LABS

TASK

01

DIVIDE THE DATA INTO TRAIN SET AND TEST SET

02

CONSTRUCT DIFFERENT REGRESSION ALGORITHMS

03

CALCULATE DIFFERENT REGRESSION METRICS

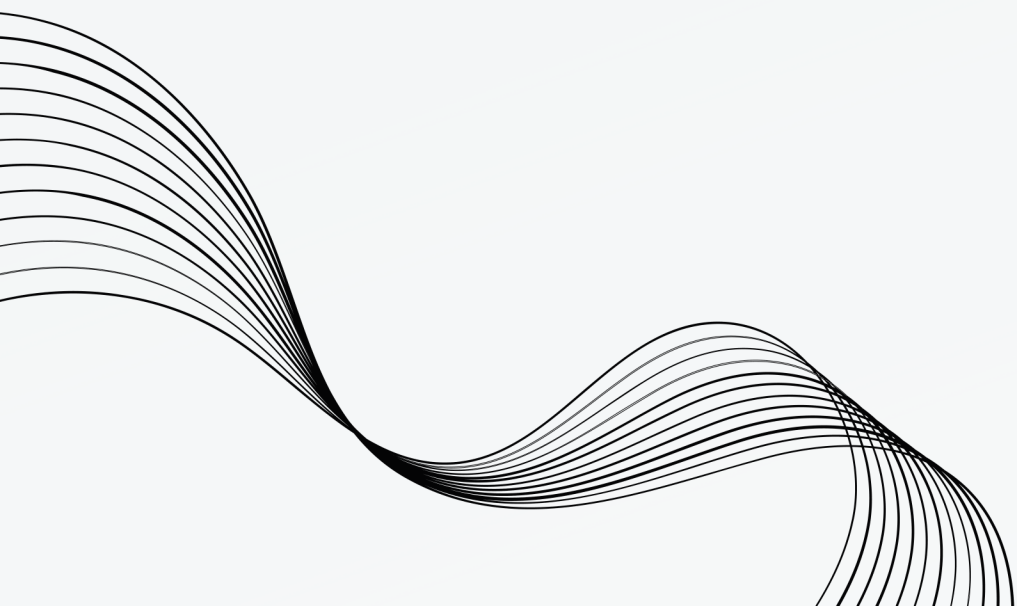
04

CHOOSE THE BEST MODEL

IMPORT LIBRARY

```
# Importing libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

✓ 5.0s



LOAD DATA & DATA PREPROCESSING

```
# Loading the dataset  
data = pd.read_csv('50_Startups.csv')
```

✓ 0.0s

```
# Data Preprocessing  
x = data[['R&D Spend', 'Administration', 'Marketing Spend']].values  
y = data['Profit'].values
```

✓ 0.0s

01

DIVIDE THE DATA INTO TRAIN SET AND TEST SET

```
# Splitting the dataset into training and testing sets  
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)  
✓ 0.0s
```

X_TRAIN

x_train			
✓ 0.0s			
	R&D Spend	Administration	Marketing Spend
33	55493.95	103057.49	214634.81
35	46014.02	85047.44	205517.64
26	75328.87	144135.98	134050.07
34	46426.07	157693.92	210797.67
18	91749.16	114175.79	294919.57
7	130298.13	145530.06	323876.68
14	119943.24	156547.42	256512.92
45	1000.23	124153.04	1903.93
48	542.05	51743.15	0.00
29	65605.48	153032.06	107138.38
15	114523.61	122616.84	261776.23
30	61994.48	115641.28	91131.24
32	63408.86	129219.61	46085.25
16	78013.11	121597.55	264346.06
42	23640.93	96189.63	148001.11
20	76253.86	113867.30	298664.47
43	15505.73	127382.30	35534.17

8	120542.52	148718.95	311613.29
13	91992.39	135495.07	252664.93
25	64664.71	139553.16	137962.62
5	131876.90	99814.71	362861.36
17	94657.16	145077.58	282574.31
40	28754.33	118546.05	172795.67
49	0.00	116983.80	45173.06
1	162597.70	151377.59	443898.53
12	93863.75	127320.38	249839.44
37	44069.95	51283.14	197029.42
24	77044.01	99281.34	140574.81
6	134615.46	147198.87	127716.82
23	67532.53	105751.03	304768.73
36	28663.76	127056.21	201126.82
21	78389.47	153773.43	299737.29
19	86419.70	153514.11	0.00
9	123334.88	108679.17	304981.62
39	38558.51	82982.09	174999.30
46	1315.46	115816.21	297114.46
3	144372.41	118671.85	383199.62
0	165349.20	136897.80	471784.10
47	0.00	135426.92	0.00
44	22177.74	154806.14	28334.72

X_TEST

x_test

✓ 0.0s

	R&D Spend	Administration	Marketing Spend
28	66051.52	182645.56	118148.20
11	100671.96	91790.61	249744.55
10	101913.08	110594.11	229160.95
41	27892.92	84710.77	164470.71
2	153441.51	101145.55	407934.54
27	72107.60	127864.55	353183.81
38	20229.59	65947.93	185265.10
31	61136.38	152701.92	88218.23
22	73994.56	122782.75	303319.26
4	142107.34	91391.77	366168.42

Y_TRAIN

y_train			
✓ 0.0s			
	R&D Spend	Administration	Marketing Spend
28	66051.52	182645.56	118148.20
11	100671.96	91790.61	249744.55
10	101913.08	110594.11	229160.95
41	27892.92	84710.77	164470.71
2	153441.51	101145.55	407934.54
27	72107.60	127864.55	353183.81
38	20229.59	65947.93	185265.10
31	61136.38	152701.92	88218.23
22	73994.56	122782.75	303319.26
4	142107.34	91391.77	366168.42

Y_TEST

y_test			
✓ 0.0s			
	R&D Spend	Administration	Marketing Spend
28	66051.52	182645.56	118148.20
11	100671.96	91790.61	249744.55
10	101913.08	110594.11	229160.95
41	27892.92	84710.77	164470.71
2	153441.51	101145.55	407934.54
27	72107.60	127864.55	353183.81
38	20229.59	65947.93	185265.10
31	61136.38	152701.92	88218.23
22	73994.56	122782.75	303319.26
4	142107.34	91391.77	366168.42

02

CONSTRUCT DIFFERENT REGRESSION ALGORITHMS

LINEAR REGRESSION

```
# Linear Regression
linear_regressor = LinearRegression()
linear_regressor.fit(x_train, y_train)
y_pred_linear = linear_regressor.predict(x_test)
y_pred_linear
```

✓ 0.0s

```
array([103901.8969696 , 132763.05993126, 133567.90370044,  72911.78976736,
       179627.92567224, 115166.64864795,  67113.5769057 ,  98154.80686776,
       114756.11555221, 169064.01408795])
```


POLYNOMIAL REGRESSION

```
# Polynomial Regression
poly_regressor = PolynomialFeatures(degree=2)
x_poly = poly_regressor.fit_transform(x_train)
lin_regressor2 = LinearRegression()
lin_regressor2.fit(x_poly, y_train)
y_pred_poly = lin_regressor2.predict(poly_regressor.transform(x_test))
y_pred_poly
```

✓ 0.0s

```
array([104055.41052512, 133038.9768166 , 132790.41574745,  72366.6582    ,
       180753.47707241, 121410.27078678,  66480.88516424, 100549.44023129,
       118950.17510935, 169529.23853235])
```


DECISION TREE REGRESSION

```
# Decision Tree Regression  
tree_regressor = DecisionTreeRegressor(random_state=0)  
tree_regressor.fit(x_train, y_train)  
y_pred_tree = tree_regressor.predict(x_test)  
y_pred_tree
```

✓ 0.0s

```
array([101004.64, 141585.52, 141585.52,  78239.91, 182901.99, 108733.99,  
       71498.49,  99937.59, 108733.99, 156991.12])
```

RANDOM FOREST REGRESSION

```
# Random Forest Regression
forest_regressor = RandomForestRegressor(n_estimators=10, random_state=0)
forest_regressor.fit(x_train, y_train)
y_pred_forest = forest_regressor.predict(x_test)
y_pred_forest
```

✓ 0.0s

```
array([102284.58 , 133599.144, 134591.343,  80593.078, 183024.901,
       115306.291,  75525.004, 100713.216, 110640.273, 172413.79 ])
```

SUPPORT VECTOR REGRESSION

```
# Support Vector Regression
sc_x = StandardScaler()
sc_y = StandardScaler()
x_train_scaled = sc_x.fit_transform(x_train)
x_test_scaled = sc_x.transform(x_test)
y_train_scaled = sc_y.fit_transform(y_train.reshape(-1,1)).ravel()
svr_regressor = SVR(kernel='rbf')
svr_regressor.fit(x_train_scaled, y_train_scaled)
y_pr = svr_regressor.predict(x_test_scaled)
y_pred_svr = sc_y.inverse_transform(y_pr.reshape(-1,1)).ravel()
y_pred_svr
```

✓ 0.0s

```
array([100160.05210776, 122549.42139322, 131502.13740292, 78872.3678589 ,
       162946.50120064, 121600.82914911, 81081.26971832, 97033.72585873,
       121069.83056306, 147031.29084299])
```

03

CALCULATE DIFFERENT REGRESSION METRICS

```
# Calculating Regression Metrics
def evaluate_model(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    return mae, mse, r2

metrics = {
    'Linear Regression': evaluate_model(y_test, y_pred_linear),
    'Polynomial Regression': evaluate_model(y_test, y_pred_poly),
    'Decision Tree Regression': evaluate_model(y_test, y_pred_tree),
    'Random Forest Regression': evaluate_model(y_test, y_pred_forest),
    'Support Vector Regression': evaluate_model(y_test, y_pred_svr)
}
metrics
```

✓ 0.0s

OUTPUT

```
{'Linear Regression': (7320.44161484813, 77506468.16885415, 0.939395591782057),  
 'Polynomial Regression': (8720.931218418915,  
 102078137.67353782,  
 0.9201823373990293),  
 'Decision Tree Regression': (4480.289000000006,  
 30129511.580150068,  
 0.9764409182568788),  
 'Random Forest Regression': (5975.3965999999955,  
 50766736.70654713,  
 0.9603041125735003),  
 'Support Vector Regression': (11569.391904650076,  
 224317169.661448,  
 0.8246003250872043)}
```


04

CHOOSE THE BEST MODEL

```
# Selecting the Best Model
best_model = min(metrics, key=lambda x: metrics[x][1]) # Choosing based on lowest MSE
✓ 0.0s
```

```
# Displaying Metrics
for model, (mae, mse, r2) in metrics.items():
    print(f"{model}: MAE={mae}, MSE={mse}, R2={r2}")

print(f"Best Model: {best_model}")
```

✓ 0.0s

```
Linear Regression: MAE=7320.44161484813, MSE=77506468.16885415, R2=0.939395591782057
Polynomial Regression: MAE=8720.931218418915, MSE=102078137.67353782, R2=0.9201823373990293
Decision Tree Regression: MAE=4480.289000000006, MSE=30129511.580150068, R2=0.9764409182568788
Random Forest Regression: MAE=5975.39659999999955, MSE=50766736.70654713, R2=0.9603041125735003
Support Vector Regression: MAE=11569.391904650076, MSE=224317169.661448, R2=0.8246003250872043
Best Model: Decision Tree Regression
```

**THANK'S FOR
WATCHING**

